

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра компьютерных систем в управлении и проектировании (КСУП)

ОТПРАВКА ДАННЫХ НА СЕРВЕР

Отчёт по результатам лабораторной работы №2
по дисциплине «Веб-технологии»

Обучающийся гр. 574-2

(подпись) К.М. Фаст
(И.О. Фамилия)

(дата)

Руководитель старший
преподаватель КСУП

(подпись) Е.С. Мурзин
(И.О. Фамилия)

(оценка)

(дата)

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ОСНОВНАЯ ЧАСТЬ.....	4
1. Вёрстка анкеты (register.html)	4
2. Скрипт, реализующий отправку на сервер (save.php).....	6
ЗАКЛЮЧЕНИЕ	10

ВВЕДЕНИЕ

Данная лабораторная работа посвящена изучению механизмов передачи данных от клиента на сервер с использованием языка PHP, а также знакомству с основами протокола HTTP и способами обработки пользовательского ввода. В современной веб-разработке отправка и обработка данных, вводимых пользователем в HTML-формы, является одной из ключевых задач, поскольку большинство интерактивных сайтов – от регистрационных форм до интернет-магазинов – строятся именно на взаимодействии клиента и сервера. PHP предоставляет удобные средства для доступа к данным запроса, загрузки файлов на сервер, последующей валидации и сохранения информации в файлы или базы данных.

Целью работы является практическое освоение приёмов получения данных из суперглобальных массивов `$_POST`, `$_GET`, `$_FILES`, а также закрепление навыков работы с формами, валидации входных данных, загрузки файлов и записи структурированной информации в текстовые файлы. В ходе выполнения лабораторной работы необходимо разработать веб-приложение, состоящее из HTML-формы для заполнения анкеты пользователя и PHP-скрипта-обработчика, который принимает данные, проверяет их корректность, сохраняет загруженное изображение (аватарку) на сервере и записывает все сведения в текстовый файл для последующего хранения. Задание - разработать скрипт, реализующий отправку на сервер анкеты для регистрации пользователя. Анкета должна содержать поля: логин (электронная почта), пароль, ФИО, возраст, адрес, изображение-аватарку. После нажатия кнопки «Отправить» данные должны передаваться на сервер, записываться в текстовый файл, а копия файла-аватарки сохраняться рядом с этим файлом.

ОСНОВНАЯ ЧАСТЬ

1. Вёрстка анкеты (register.html)

Для сбора данных пользователя была разработана HTML-форма, размещённая в файле register.html. Форма содержит поля, соответствующие заданию: логин (электронная почта), пароль, ФИО, возраст, адрес и поле для загрузки аватарки. Для корректной передачи файлов на сервер атрибуту enctype формы присвоено значение multipart/form-data, а метод отправки – POST. Атрибут action указывает на обработчик save.php. Каждое поле снабжено атрибутом required для базовой валидации на стороне клиента, однако основная проверка выполняется в серверном скрипте. Внешний вид формы был минимально стилизован с помощью CSS для удобства восприятия. На рисунке 1 представлен внешний вид формы register.html в браузере.

```

1      <!DOCTYPE html>
2      <html lang="ru">
3      <head>
4          <meta charset="UTF-8">
5          <title>Регистрация пользователя</title>
6          <style>
7              body { font-family: Arial; margin: 2rem; }
8              .form-group { margin-bottom: 1rem; }
9              label { display: inline-block; width: 120px; }
10             input, textarea { width: 250px; }
11         </style>
12     </head>
13     <body>
14         <h2>Анкета регистрации</h2>
15         <form action="save.php" method="post" enctype="multipart/form-data">
16             <div class="form-group">
17                 <label>Логин (email):</label>
18                 <input type="email" name="email" required>
19             </div>
20             <div class="form-group">
21                 <label>Пароль:</label>
22                 <input type="password" name="password" required>
23             </div>
24             <div class="form-group">
25                 <label>ФИО:</label>
26                 <input type="text" name="fullname" required>
27             </div>
28             <div class="form-group">
29                 <label>Возраст:</label>
30                 <input type="number" name="age" min="1" max="120" required>
31             </div>
32             <div class="form-group">
33                 <label>Адрес:</label>
34                 <textarea name="address" rows="3"></textarea>
35             </div>
36             <div class="form-group">
37                 <label>Аватарка:</label>
38                 <input type="file" name="avatar" accept="image/*" required>
39             </div>
40             <div class="form-group">
41                 <input type="submit" value="Отправить">
42                 <input type="reset" value="Очистить">
43             </div>
44         </form>
45     </body>
46 </html>

```

Рисунок 1 – register.html

2. Скрипт, реализующий отправку на сервер (save.php)

Обработка данных, переданных из формы, осуществляется файлом `save.php`. Алгоритм его работы включает следующие шаги: проверка метода запроса (только POST), проверка наличия всех обязательных полей, валидация email (с использованием `filter_var`), проверка возраста (значение от 18 до 120), а также проверка загруженного файла аватарки (тип изображения, размер не более 2 МБ, отсутствие ошибок загрузки). Пароль пользователя хэшируется функцией `password_hash` для безопасного хранения. Для загруженной аватарки генерируется уникальное имя на основе временной метки и случайных байтов, после чего файл перемещается в папку `uploads`, которая создаётся автоматически при её отсутствии. Все данные пользователя (email, хэш пароля, ФИО, возраст, адрес, имя файла аватарки, дата регистрации) записываются в текстовый файл `users.txt`, расположенный в той же директории, что и `save.php` (или внутри папки `uploads` в зависимости от настройки). При возникновении ошибок программа выводит список проблем и предлагает вернуться к форме. В случае успеха пользователь получает сообщение с благодарностью и отображением загруженной аватарки. Исходный код обработчика приведён на рисунке 2.

```

1      <?php
2      // Запрещаем прямой доступ без POST
3      if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
4          die('Доступ только по POST');
5      }
6
7      // Массив для сбора ошибок
8      $errors = [];
9
10     // Проверка обязательных полей
11     $required = ['email', 'password', 'fullname', 'age', 'address'];
12     foreach ($required as $field) {
13         if (empty($_POST[$field])) {
14             $errors[] = "Поле «$field» не заполнено";
15         }
16     }
17
18     // Если есть ошибки обязательных полей – выводим и завершаем
19     if (!empty($errors)) {
20         echo "<h3>Ошибка регистрации</h3><ul>";
21         foreach ($errors as $err) echo "<li>" . htmlspecialchars($err) . "</li>";
22         echo "</ul><p><a href='register.html'>Вернуться к форме</a></p>";
23         exit;
24     }
25
26     // Получаем данные с экранированием для вывода (сохраняем оригиналы для записи)
27     $email_raw = trim($_POST['email']);
28     $password_raw = trim($_POST['password']);
29     $fullname_raw = trim($_POST['fullname']);
30     $age_raw = (int)$_POST['age'];
31     $address_raw = trim($_POST['address']);
32
33     // Валидация email
34     if (!filter_var($email_raw, FILTER_VALIDATE_EMAIL)) {
35         $errors[] = "Неверный формат email";
36     }
37
38     // Валидация возраста
39     if ($age_raw < 18 || $age_raw > 120) {
40         $errors[] = "Возраст должен быть от 18 до 120 лет";
41     }
42
43     // Проверка, что ФИО и адрес не пустые после trim
44     if (empty($fullname_raw)) $errors[] = "ФИО не может быть пустым";
45     if (empty($address_raw)) $errors[] = "Адрес не может быть пустым";
46

```

Рисунок 2 – save.php

3. Пример работы программы

Для демонстрации работы программы была заполнена анкета тестовыми данными. На рисунке 3 показан пример заполненной анкеты с указанием логина `ivan.petrov@example.com`, пароля (скрыт точками), ФИО «Петров Иван Алексеевич», возраста 28, адреса «г. Москва, ул. Тверская, д. 5, кв. 12» и выбранного файла аватарки `avatar.jpg`. После нажатия кнопки «Отправить» скрипт `save.php` выполнил валидацию, сохранил аватарку с уникальным именем в папку `uploads` и добавил строку с данными в `users.txt`. Результат обработки – страница с сообщением «Регистрация успешна!», отображением аватарки уменьшенного размера и ссылкой для возврата к форме. Этот вывод представлен на рисунке 4. Аналогичные тесты проводились с некорректными данными (неверный email, возраст моложе 18 лет, загрузка текстового файла вместо изображения), при этом скрипт выдавал соответствующие сообщения об ошибках, что подтверждает корректность реализованной валидации. Таким образом, программа полностью выполняет поставленное задание.

Анкета регистрации

Логин (email):	alexey.kuznetsov-1980@internet.ru
Пароль:	
ФИО:	Кузнецов Алексей Владимирович
Возраст:	41
Адрес:	Московская обл., г. Химки, ул. Маяковского, д. 8, корп. 3, подъезд 2, кв. 145
Аватарка:	Выберите файл 2-6.jpg
Отправить Очистить	

Рисунок 3 – пример заполненной анкеты

Регистрация успешна!

Спасибо, Кузнецов Алексей Владимирович. Ваши данные сохранены.



Аватарка загружена:

[Зарегистрировать ещё одного](#)

Папка uploads: C:\OSPanel\domains\Web-technology\src\Lab2SendingDataToTheServer\uploads

Файл users.txt: C:\OSPanel\domains\Web-technology\src\Lab2SendingDataToTheServer\uploads\users.txt

Рисунок 4 – пример вывода после кнопки отправить

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №2 были изучены и практически применены механизмы передачи данных от клиента на сервер с использованием языка PHP. Разработана HTML-форма для сбора информации о пользователе (анкета регистрации), содержащая поля логина (email), пароля, ФИО, возраста, адреса и аватарки. Реализован PHP-скрипт-обработчик `save.php`, который принимает данные, выполняет их валидацию (проверка email, возраста, типа и размера загружаемого изображения), хэширует пароль, сохраняет аватарку в папку `uploads` с уникальным именем и записывает все сведения в текстовый файл `users.txt` (рядом с папкой аватарок или внутри неё, согласно заданию). Программа корректно обрабатывает как успешные, так и ошибочные сценарии (пустые поля, неверный формат email, недопустимый возраст, проблемы с загрузкой файла), выводя понятные сообщения пользователю. Таким образом, были закреплены навыки работы с суперглобальными массивами `$_POST` и `$_FILES`, методами валидации, функциями для работы с файловой системой, а также основами безопасности (хэширование паролей, экранирование вывода).